

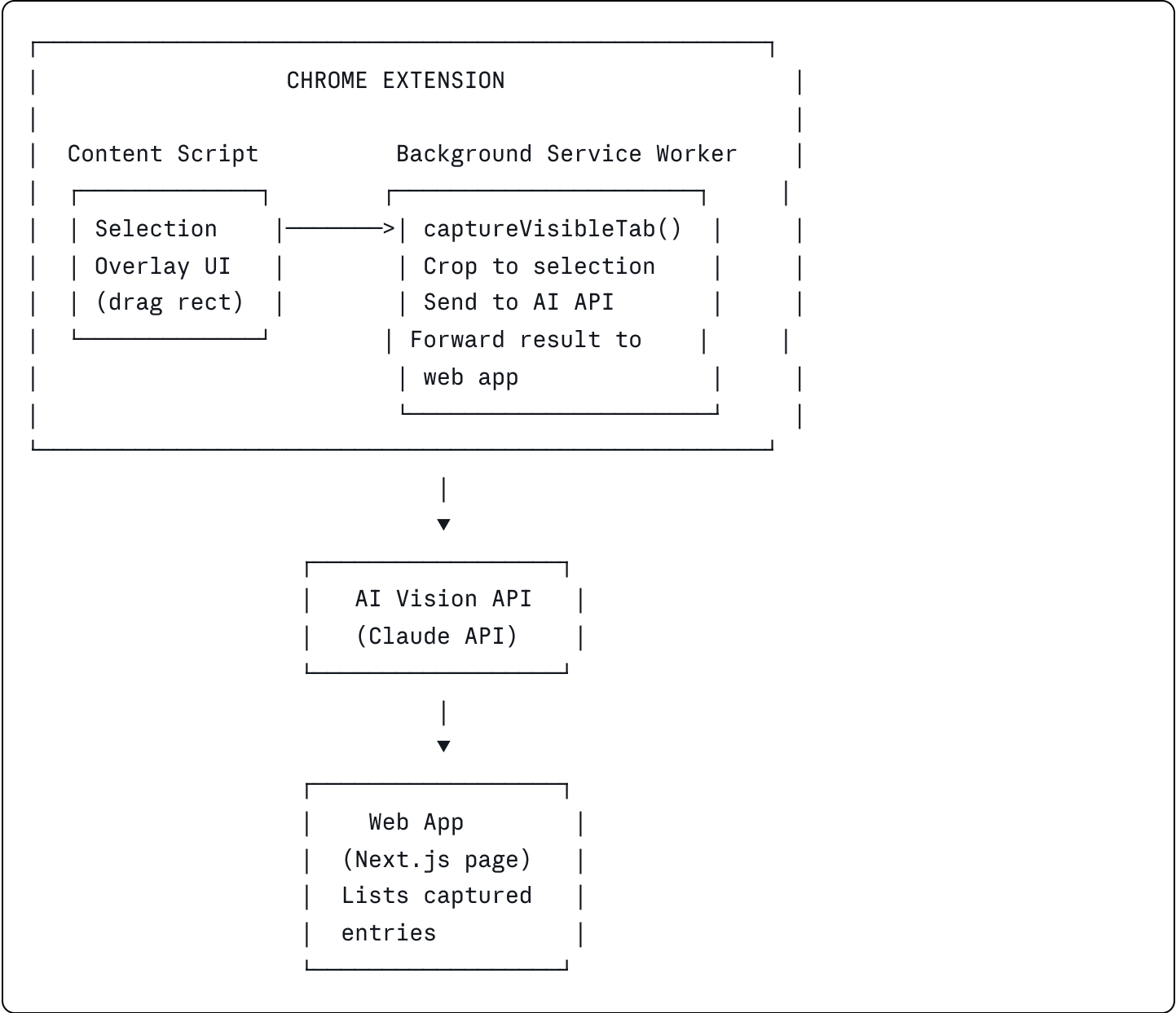
ATNX Phase 1 — Chrome Extension + Web App Build Instructions

Project Overview

Build a Chrome extension that lets users select an area of any webpage, captures a screenshot of that region, sends it to an AI vision model for analysis, and displays the extracted data on a simple web app page.

This is the MVP entry point for Attention X (ATNX), a platform where users trade on meme/trend virality. The extension reduces "time to bet" by letting users capture anything on their screen and instantly pipe it through AI for identification and data extraction.

Architecture Overview



Tech Stack

- **Chrome Extension:** Manifest V3, vanilla JavaScript
 - **Web App:** Next.js 14 (App Router)
 - **AI Model:** Anthropic Claude API (claude-sonnet-4-20250514) with vision
 - **Styling:** Tailwind CSS
 - **State/Storage:** localStorage for MVP (web app side), chrome.storage.local (extension side)
-

Part 1: Chrome Extension

File Structure

```
atnx-extension/  
├─ manifest.json  
├─ background.js          # Service worker  
├─ content.js             # Selection overlay logic  
├─ content.css            # Overlay styling  
├─ popup.html             # Extension popup UI  
├─ popup.js               # Popup logic  
├─ popup.css              # Popup styling  
└─ icons/  
    ├─ icon16.png  
    ├─ icon48.png  
    └─ icon128.png
```

manifest.json

Use Manifest V3. Required permissions:

```
json
```

```
{
  "manifest_version": 3,
  "name": "ATNX Capture",
  "version": "1.0.0",
  "description": "Capture anything on screen, identify it with AI",
  "permissions": [
    "activeTab",
    "scripting",
    "storage"
  ],
  "host_permissions": [
    "<all_urls>"
  ],
  "background": {
    "service_worker": "background.js"
  },
  "action": {
    "default_popup": "popup.html",
    "default_icon": {
      "16": "icons/icon16.png",
      "48": "icons/icon48.png",
      "128": "icons/icon128.png"
    }
  },
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["content.js"],
      "css": ["content.css"],
      "run_at": "document_idle"
    }
  ],
  "commands": {
    "activate-capture": {
      "suggested_key": {
        "default": "Ctrl+Shift+X",
        "mac": "Command+Shift+X"
      },
      "description": "Activate screen capture selection"
    }
  }
}
```

content.js — Selection Overlay

This is the core UX. Must implement:

1. **Activation:** Listen for message from background.js (triggered by hotkey `Ctrl+Shift+X` or popup button click). When activated, enter selection mode.

2. **Selection Mode:**

- Create a full-screen transparent overlay div (`position: fixed; top: 0; left: 0; width: 100vw; height: 100vh; z-index: 2147483647`)
- Change cursor to `crosshair`
- On `mousedown`: record start coordinates (clientX, clientY)
- On `mousemove` (while dragging): draw a visible selection rectangle. Use a semi-transparent colored border (suggest  `#00FF66`) with 0.15 alpha fill, 2px solid border — cyberpunk/ATNX brand green)
- On `mouseup`: record end coordinates, calculate the selection rectangle (x, y, width, height)
- Send selection coordinates to background.js via `chrome.runtime.sendMessage`
- Remove the overlay

3. **Cancel:** If user presses `Escape` during selection, cancel and remove overlay.

4. **Edge Cases:**

- Minimum selection size check (at least 20x20 pixels) — if smaller, ignore
- Prevent default click behavior while overlay is active
- Handle scrolled pages correctly (use clientX/clientY not pageX/pageY since `captureVisibleTab` only captures the viewport)

javascript

```
// Rough structure - implement fully

let isSelecting = false;
let startX, startY;
let overlay, selectionBox;

chrome.runtime.onMessage.addListener((msg) => {
  if (msg.action === 'activate-capture') {
    activateSelection();
  }
});

function activateSelection() {
  // Create overlay
  // Attach mousedown, mousemove, mouseup, keydown listeners
  // On completion, send coordinates to background
}

function sendSelection(rect) {
  chrome.runtime.sendMessage({
    action: 'capture-region',
    rect: {
      x: rect.x,
      y: rect.y,
      width: rect.width,
      height: rect.height
    },
    pageUrl: window.location.href,
    pageTitle: document.title
  });
}
```

content.css — Overlay Styles

css

```

/*
  All styles should use a unique prefix (e.g., .atnx-*)
  to avoid collisions with page styles.
  Use !important sparingly but as needed since we're injecting into unknown pages.
*/

.atnx-overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100vw;
  height: 100vh;
  z-index: 2147483647;
  cursor: crosshair;
  background: rgba(0, 0, 0, 0.1);
}

.atnx-selection {
  position: fixed;
  border: 2px solid #00FF66;
  background: rgba(0, 255, 102, 0.1);
  z-index: 2147483647;
  pointer-events: none;
}

.atnx-tooltip {
  /* Small tooltip near cursor showing "Drag to select" or dimensions */
  position: fixed;
  background: #1A1A2E;
  color: #00FF66;
  padding: 4px 8px;
  border-radius: 4px;
  font-family: monospace;
  font-size: 12px;
  z-index: 2147483647;
  pointer-events: none;
}

```

background.js — Service Worker

Handles:

1. **Hotkey command:** Listen for `chrome.commands.onCommand` for `activate-capture`. When triggered, send message to content script to activate selection.
2. **Screen capture:** When receiving `capture-region` from content script:
 - Call `chrome.tabs.captureVisibleTab(null, { format: 'png' })` to get full viewport screenshot as data URL
 - Crop the image to the selection rectangle using an `OffscreenCanvas` (Manifest V3 service workers don't have DOM, so use `OffscreenCanvas` or create an offscreen document)
 - Convert cropped image to base64
3. **AI API call:** Send the cropped screenshot to Claude API for analysis.
4. **Forward results:** Send the AI response to the web app.

javascript

```
// Command listener
chrome.commands.onCommand.addListener((command) => {
  if (command === 'activate-capture') {
    chrome.tabs.query({ active: true, currentWindow: true }, (tabs) => {
      chrome.tabs.sendMessage(tabs[0].id, { action: 'activate-capture' });
    });
  }
});

// Message listener
chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
  if (msg.action === 'capture-region') {
    handleCapture(msg, sender.tab);
  }
});

async function handleCapture(msg, tab) {
  // 1. Capture visible tab
  const dataUrl = await chrome.tabs.captureVisibleTab(null, { format: 'png' });

  // 2. Crop to selection
  // NOTE: Service workers can't use Canvas directly.
  // Option A: Use chrome.offscreen API to create an offscreen document for canvas
  // Option B: Send the full screenshot + coordinates to the web app and let it crop
  // Recommend Option A for cleaner separation

  const croppedBase64 = await cropImage(dataUrl, msg.rect);

  // 3. Send to AI
  const analysis = await analyzeWithAI(croppedBase64, msg.pageUrl, msg.pageTitle);

  // 4. Store result and/or send to web app
  await storeResult({
    id: Date.now().toString(),
    timestamp: new Date().toISOString(),
    screenshot: croppedBase64,
    pageUrl: msg.pageUrl,
    pageTitle: msg.pageTitle,
    analysis: analysis
  });
}
```


Image Cropping (Offscreen Document)

Since Manifest V3 service workers can't use Canvas, create an offscreen document:

```
atnx-extension/  
├─ offscreen.html  
└─ offscreen.js
```

offscreen.html:

```
html  
  
<!DOCTYPE html>  
<html><body><script src="offscreen.js"></script></body></html>
```

offscreen.js: Receives the full screenshot data URL and crop coordinates, uses Canvas to crop, returns the cropped data URL.

In **background.js**, create the offscreen document when needed:

```
javascript  
  
await chrome.offscreen.createDocument({  
  url: 'offscreen.html',  
  reasons: ['CANVAS'],  
  justification: 'Crop screenshot to selection area'  
});
```

AI API Call

Call the Anthropic Claude API with the cropped image:

```
javascript
```

```

async function analyzeWithAI(base64Image, pageUrl, pageTitle) {
  // Get API key from storage (user enters in popup settings)
  const { apiKey } = await chrome.storage.local.get('apiKey');

  const response = await fetch('https://api.anthropic.com/v1/messages', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'x-api-key': apiKey,
      'anthropic-version': '2023-06-01',
      'anthropic-dangerous-direct-browser-access': 'true'
    },
    body: JSON.stringify({
      model: 'claude-sonnet-4-20250514',
      max_tokens: 1024,
      messages: [{
        role: 'user',
        content: [
          {
            type: 'image',
            source: {
              type: 'base64',
              media_type: 'image/png',
              data: base64Image // raw base64, no data:image/png;base64, prefix
            }
          },
          {
            type: 'text',
            text: `Analyze this screenshot captured from ${pageUrl} (${pageTitle}).`
          }
        ]
      }]
    })
  });
}

```

Identify the main subject/content and extract the following information. Respond ONLY

```

{
  "type": "meme" | "trend" | "person" | "brand" | "event" | "other",
  "name": "The primary name/identifier of the subject",
  "description": "Brief 1-2 sentence description of what this is",
  "category": "e.g. crypto, politics, entertainment, tech, sports, culture",
  "platforms_detected": ["list of any social platforms visible in the screenshot"],
  "metrics_detected": {
    "any visible numbers like views, likes, followers, subscribers etc"
  },
  "sentiment": "positive" | "negative" | "neutral" | "mixed",
  "virality_signals": "Brief assessment of any virality indicators visible",
}

```

```

    "raw_text": "Any readable text extracted from the image"
  }`
    }
  ]
}]]
  })
});

const data = await response.json();

// Parse the JSON response from Claude
const textContent = data.content.find(c => c.type === 'text')?.text;
try {
  return JSON.parse(textContent.replace(/``json|``/g, '').trim());
} catch {
  return { raw_response: textContent, parse_error: true };
}
}

```

popup.html / popup.js

Simple popup with:

1. **API Key input:** Text field for user to enter their Anthropic API key. Save to `chrome.storage.local`. Show saved/unsaved state.
2. **Capture button:** Big green button that triggers the selection mode (sends message to content script).
3. **Status indicator:** Shows "Ready", "Capturing...", "Analyzing...", "Done!" states.
4. **Recent captures count:** Shows how many captures are stored.
5. **"Open Dashboard" link:** Opens the web app in a new tab.

Style with the ATNX brand: dark background (`#0D0D1A`), green accent (`#00FF66`), monospace font.

Data Flow: Extension → Web App

For MVP, use one of these approaches (implement the simplest one that works):

Option A — Local Storage Bridge (Simplest): The extension stores results in `chrome.storage.local`. The web app (when running on localhost or a known domain) communicates with the extension via `chrome.runtime.sendMessage` using `externally_connectable` in manifest:

```
json
```

```
"externally_connectable": {  
  "matches": ["http://localhost:3000/*", "https://atnx.app/*"]  
}
```

The web app can then request stored captures from the extension.

Option B — POST to Local API (Recommended for MVP): The extension POSTs the analysis result to the Next.js web app's API route:

```
javascript
```

```
// In background.js after getting AI analysis  
await fetch('http://localhost:3000/api/captures', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify(result)  
});
```

This is simpler and doesn't require `externally_connectable` configuration. Go with this.

Part 2: Web App (Next.js)

File Structure

```
atnx-web/  
├─ package.json  
├─ next.config.js  
├─ tailwind.config.js  
├─ app/  
│   ├─ layout.tsx  
│   ├─ page.tsx           # Main dashboard - lists all captures  
│   ├─ globals.css  
│   └─ api/  
│       └─ captures/  
│           └─ route.ts    # POST: receive new capture, GET: return all  
captures  
└─ lib/  
    └─ store.ts           # Simple in-memory or file-based store for MVP
```

API Route — `/api/captures/route.ts`

typescript

```
// Simple in-memory store for MVP
// Later: replace with database

let captures: any[] = [];

export async function POST(request: Request) {
  const data = await request.json();

  const capture = {
    id: data.id || Date.now().toString(),
    timestamp: data.timestamp || new Date().toISOString(),
    pageUrl: data.pageUrl,
    pageTitle: data.pageTitle,
    screenshot: data.screenshot, // base64 – store for now, optimize later
    analysis: data.analysis
  };

  captures.unshift(capture); // newest first

  return Response.json({ success: true, id: capture.id });
}

export async function GET() {
  return Response.json({ captures });
}
```

Main Page — app/page.tsx

Simple dashboard that:

1. Fetches captures from `/api/captures` on load (poll every 5 seconds for new entries)
2. Displays each capture as a card in a list (newest first)
3. Each card shows:
 - Timestamp
 - Source URL
 - Screenshot thumbnail (small)
 - AI analysis results: type, name, description, category, sentiment
 - Any detected metrics
 - Raw extracted text (collapsible)

DO NOT over-engineer the UI. This is a functional MVP to demonstrate the pipeline works. Keep it simple and clean.

Styling

Use Tailwind with the ATNX brand:

```
javascript

// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        'atnx': {
          'bg': '#0D0D1A',
          'surface': '#1A1A2E',
          'green': '#00FF66',
          'green-dark': '#00CC52',
          'text': '#E0E0E0',
          'text-muted': '#888888',
        }
      },
    },
    fontFamily: {
      'mono': ['JetBrains Mono', 'Fira Code', 'monospace']
    }
  }
}
```

Dark theme, monospace font, green accents. Cyberpunk terminal aesthetic consistent with the ATNX brand.

Page Layout

ATNX		[n] Captures	
Attention Exchange			
 [thumbnail]		TYPE: meme	
		NAME: Quantum Cats	
		CATEGORY: culture	

```
SENTIMENT: positive
SOURCE: twitter.com/...
CAPTURED: 2 min ago

"Emerging cat meme with
  high engagement signals"

  ▶ Raw text
  ▶ Detected metrics

... next capture card ...
```

Extension Setup

1. `cd atnx-extension`

2. Open Chrome → `chrome://extensions`

3. Enable Developer Mode

4. Click "Load unpacked" → select the `atnx-extension` folder

5. Click extension icon → enter Anthropic API key → save

6. Press **Ctrl+Shift+X** on any page to test capture

Web App Setup

1. `cd atnx-web`2. `npm install`

3. `npm run dev` (runs on localhost:3000)

4. Make a capture with the extension

5. Check the web app — the capture should appear in the list

Implementation Priority

Build in this exact order:

1. **Selection overlay** (content.js + content.css) — get the drag-to-select working first
 2. **Screenshot capture** (background.js) — capture and crop the visible tab
 3. **AI analysis** (background.js) — send to Claude, get structured response
 4. **Web app API route** — receive and store captures
 5. **Web app dashboard** — display the captures
 6. **Popup UI** — API key input, capture button, status
 7. **Polish** — error handling, loading states, edge cases
-

Important Notes

- **API Key Security:** For MVP, storing the API key in chrome.storage.local is fine. For production, the API call should go through a backend server so the key isn't exposed in the extension.
 - **CORS:** The extension's service worker can make cross-origin requests to the Anthropic API without CORS issues. The POST to localhost:3000 may need CORS headers on the Next.js API route — add appropriate headers.
 - **Image Size:** Full viewport screenshots can be large. The crop step is important to keep the AI API payload reasonable. Also strip the `data:image/png;base64,` prefix before sending to Claude.
 - **Rate Limiting:** Each capture = 1 API call. For MVP this is fine. Later, consider batching or caching.
 - **Manifest V3 Service Worker Lifecycle:** Service workers can go idle. Use chrome.storage for any state that needs to persist. Don't rely on global variables in background.js across long periods.
 - **Testing:** Test on various sites — Twitter, Reddit, YouTube, news sites, Google Trends. Make sure the overlay doesn't break on sites with complex z-index stacking or iframes.
-

Future Enhancements (NOT for Phase 1)

These are noted for context but should NOT be built now:

- Persistent database (PostgreSQL/Supabase) replacing in-memory store
 - User authentication
 - Virality index calculation
 - Market creation from captures
 - Real-time chart display
 - Entry categorization and deduplication
 - Contributor scoring system
 - Mobile-responsive web app
 - Published Chrome Web Store listing
-

Success Criteria for Phase 1

The build is done when:

1. ☒ User presses Ctrl+Shift+X on any webpage
2. ☒ A selection overlay appears with crosshair cursor
3. ☒ User drags a rectangle to select an area
4. ☒ The selected area is captured and cropped
5. ☒ The cropped image is sent to Claude for analysis
6. ☒ Structured JSON data is returned (type, name, category, sentiment, etc.)
7. ☒ The result appears on the web app dashboard
8. ☒ Multiple captures accumulate in a list (newest first)
9. ☒ The whole flow takes under 30 seconds end to end